

BÁO CÁO THỰC HÀNH

Thực hành Nhập môn Mạng máy tính

Lab 02: Mật mã học hiện đại

GVTH: Nguyễn Ngọc Trường

Ngày báo cáo: XX/03/2026

Nhóm 05 – NT101.Q22.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24522019	Nguyễn Phú Vinh	Nhiệm vụ 2.1, 2.2, 2.3	50%
24522028	Võ Lê Thành Vinh	Nhiệm vụ 2.4, 2.5, 2.6	50%

2. Nội dung thực hành

Nhiệm vụ 2.1 (*)

Dựa trên Định lý 1.2, hãy hoàn thiện hàm **feistel_round**. Sau đó, thực hiện mã hóa hai bản rõ $M1 = 0xAB$ và $M2 = 0xAC$ (chỉ khác nhau 1 bit cuối).

Yêu cầu: In giá trị (L, R) sau mỗi vòng của cả hai quá trình và nhận xét về sự khác biệt giữa chúng tăng dần qua từng vòng như thế nào.

Ta có source code:

```
def F(right, subkey):
    return (right ^ subkey) & 0x0F

def feistel_round(L_in, R_in, subkey):
    # Nửa trái mới nhận giá trị của nửa phải cũ
    L_out = R_in

    # Nửa phải mới là phép XOR giữa nửa trái cũ và kết quả hàm F
    R_out = L_in ^ F(R_in, subkey)

    return L_out, R_out

def track_avalanche(msg, key):
    L, R = (msg >> 4) & 0x0F, msg & 0x0F
    subkeys = [key & 0x0F, (key >> 4) & 0x0F, (key + 1) & 0x0F, (key + 2) & 0x0F]
    print(f'Khởi tạo: L={format(L, '04b')}, R={format(R, '04b')}')

    for i in range(4):
        L, R = feistel_round(L, R, subkeys[i])
        print(f'Vòng {i+1}: L={format(L, '04b')}, R={format(R, '04b')}')

    return (L << 4) | R

# Chạy thử với 2 bản rõ
print("--- Mã hóa M1 ---")
track_avalanche(0xAB, 0x12)

print("--- Mã hóa M2 ---")
track_avalanche(0xAC, 0x12)
```

Hàm giả ngẫu nhiên $F(\text{right}, \text{subkey})$: Hàm này nhận đầu vào là nửa phải của khối dữ liệu và khóa con của vòng hiện tại. Phép toán được thực hiện là XOR ($\wedge$) giữa dữ liệu và khóa con, sau đó dùng phép AND bitwise ($\& 0x0F$) để đảm bảo kết quả trả về luôn nằm gọn trong 4 bit (chuẩn hóa kích thước).

Hàm một vòng mã hóa feistel_round(L_in, R_in, subkey): Đây là trái tim của thuật toán. Trong mỗi vòng, dữ liệu được chia làm hai nửa trái (LE) và phải (RE). Theo cấu trúc Feistel, dữ liệu được cập nhật theo công thức:

$LE_i = RE_{i-1}$: Nửa trái của vòng mới chính là nửa phải của vòng trước đó.

$RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i)$: Nửa phải của vòng mới được tạo ra bằng cách lấy nửa trái của vòng trước XOR với kết quả của hàm F .

Hàm track_avalanche(msg, key): Đầu tiên, hàm tách bản rõ 8-bit (ví dụ 0xAB) thành 2 khối 4-bit: L (4 bit đầu) và R (4 bit cuối) thông qua phép dịch bit $\gg$ và AND. Tiếp theo, hàm vòng lặp for chạy 4 lần, gọi feistel_round và in ra kết quả L, R dưới dạng nhị phân sau mỗi vòng để dễ dàng quan sát sự dịch chuyển của các bit.

Kết quả khi chạy code:

```

... \TH\Lab02_ModernCryptography  P main ?  14:16
• ) python .\task2_1.py
--- Mã hóa M1 ---
Khởi tạo: L=1010, R=1011
Vòng 1: L=1011, R=0011
Vòng 2: L=0011, R=1001
Vòng 3: L=1001, R=1001
Vòng 2: L=0011, R=1001
Vòng 3: L=1001, R=1001
Vòng 4: L=1001, R=0100
--- Mã hóa M2 ---
Khởi tạo: L=1010, R=1100
Vòng 1: L=1100, R=0100
Vòng 2: L=0100, R=1001
Vòng 3: L=1001, R=1110
Vòng 4: L=1110, R=0011

```

Giải thích kết quả chạy:

Cấu trúc Feistel có một quy luật bất di bất dịch cho mỗi vòng: **Nửa trái của vòng mới luôn nhận giá trị từ nửa phải của vòng cũ** ($LE_i = RE_{i-1}$).

Nhìn vào kết quả của **M1**, bạn sẽ thấy quy luật này cực kỳ rõ ràng:

- Khởi tạo: $R=1011 \rightarrow$ Vòng 1: $L=1011$
- Vòng 1: $R=0011 \rightarrow$ Vòng 2: $L=0011$
- Vòng 2: $R=1001 \rightarrow$ Vòng 3: $L=1001$

Trong khi đó, nửa phải mới (R_{new}) được tính toán phức tạp hơn bằng cách lấy nửa trái cũ XOR với kết quả của hàm F . Đây chính là nơi tạo ra sự xáo trộn dữ liệu.

Nhiệm vụ 2.2 (*)

Sử dụng thư viện **pycryptodome**, hãy viết script mã hóa chuỗi văn bản có tính lặp lại cao: "UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_". Thực hiện mã hóa bằng cả hai chế độ

AES-ECB và **AES-CBC** (với IV cố định).

- In kết quả dưới dạng Hexadecimal và so sánh các khối 16-byte của bản mã.
- Bạn hãy tìm hiểu, thử nghiệm tất cả các Mode of Operation khác, và đưa ra nhận xét.

Source code của bài:

```
from Crypto.Cipher import AES

# Khóa 16 bytes (128 bits)
key = b'1234567890123456'

# Bản rõ 32 bytes gồm 2 khối 16 bytes giống hệt nhau
plaintext = b"UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_"

print("--- THÔNG TIN ĐẦU VÀO ---")
print(f"Block 1: {plaintext[:16]}")
print(f"Block 2: {plaintext[16:]}")

# 1. Chế độ AES-ECB
cipher_ecb = AES.new(key, AES.MODE_ECB)
ct_ecb = cipher_ecb.encrypt(plaintext)

# 2. Chế độ AES-CBC
# Cần thêm Vector khởi tạo (IV) độ dài 16 bytes
iv = b'0000000000000000'
cipher_cbc = AES.new(key, AES.MODE_CBC, iv)
ct_cbc = cipher_cbc.encrypt(plaintext)

# Hàm phụ để in format 2 khối rõ ràng (32 ký tự hex = 16 bytes)
def print_blocks(ct_hex):
    return f"Block 1: {ct_hex[:32]}\nBlock 2: {ct_hex[32:]}"

print("--- KẾT QUẢ MÃ HÓA AES-ECB ---")
print(print_blocks(ct_ecb.hex()))
print()

print("--- KẾT QUẢ MÃ HÓA AES-CBC ---")
print(print_blocks(ct_cbc.hex()))
```

Phân tích kết quả mã hóa ECB vs CBC

- **Với AES-ECB:** Bản mã thu được sẽ có Block 1 và Block 2 **giống hệt nhau**. Nguyên nhân là do chế độ ECB mã hóa từng khối dữ liệu một cách độc lập bằng cùng một khóa. Vì Block 1 == Block 2, hàm mã hóa sinh ra cùng một kết quả.
 - *Nhận xét:* Điều này làm lộ cấu trúc của bản rõ (kẻ tấn công có thể biết dữ liệu đang lặp lại). Đây là lý do ECB không an toàn cho dữ liệu lớn.
- **Với AES-CBC:** Bản mã thu được sẽ có Block 1 và Block 2 **hoàn toàn khác nhau**, mặc dù bản rõ ban đầu giống hệt nhau.

- *Nguyên lý hoạt động*: Ở chế độ CBC, trước khi đưa vào mã hóa, khối bản rõ hiện tại sẽ được XOR với khối bản mã trước đó (với khối đầu tiên thì XOR với Vector khởi tạo IV). Cụ thể:
 - Mã hóa Block 1: $\text{Cipher_1} = \text{Encrypt}(\text{Plain_1} \oplus \text{IV}, \text{Key})$
 - Mã hóa Block 2: $\text{Cipher_2} = \text{Encrypt}(\text{Plain_2} \oplus \text{Cipher_1}, \text{Key})$
- *Nhận xét*: Tính chất dây chuyền (Chaining) này phá vỡ hoàn toàn các mẫu lặp lại của bản rõ, che giấu cấu trúc dữ liệu xuất sắc.

Kết quả khi chạy:

```

... \TH\Lab02_ModernCryptography  ? main ?  14:43
python .\task2_2.py
--- THÔNG TIN ĐẦU VÀO ---
Block 1: b'UIT_LAB_UIT_LAB_'
Block 2: b'UIT_LAB_UIT_LAB_'

--- KẾT QUẢ MÃ HÓA AES-ECB ---
Block 1: 02b0647de210da452e3bdcadc415814e
Block 2: 02b0647de210da452e3bdcadc415814e

--- KẾT QUẢ MÃ HÓA AES-CBC ---
Block 1: 47747530a46ebd156c1da8763c54be1b
Block 2: 46807a7ec513e5e69d9256881a0f0c09

```

Tìm hiểu các Mode of Operation khác:

CFB (Cipher Feedback) & OFB (Output Feedback): Biến mật mã khối thành mật mã luồng (Stream cipher). Thay vì mã hóa trực tiếp bản rõ, hệ thống mã hóa IV/khối trước đó để tạo ra "dòng khóa" (keystream), sau đó XOR với bản rõ. Rất tốt cho truyền dữ liệu liên tục hoặc kênh nhiễu.

CTR (Counter): Sử dụng một bộ đếm (counter) tăng dần và mã hóa bộ đếm này, sau đó XOR với bản rõ. Ưu điểm lớn nhất là có thể mã hóa/giải mã song song (parallel) giúp tăng tốc độ xử lý, rất phổ biến hiện nay.

Nhiệm vụ 2.3 (*)

Viết script mã hóa hai chuỗi "STAYHOME" và "STAYHOMA" bằng thuật toán DES. Chuyển kết quả bản mã sang dạng chuỗi nhị phân và đếm số lượng bit khác nhau (Hamming Distance). Thay đổi key lần lượt với mã số sinh viên trong nhóm, tính toán tỷ lệ phần trăm thay đổi bit với các lần thay đổi và đưa ra nhận xét.

Source code của bài:

```

from Crypto.Cipher import DES

def avalanche_test(key_bytes):
    # Đảm bảo key phải đúng 8 bytes cho thuật toán DES
    if len(key_bytes) != 8:
        print(f'Lỗi: Key '{key_bytes}' không đủ 8 bytes. Vui lòng thêm ký tự để đủ 8 bytes.")
    return

```

```

p1 = b'STAYHOME'
p2 = b'STAYHOMA' # Chỉ khác 1 ký tự cuối (E và A)

# 1. Khởi tạo thuật toán DES chế độ ECB
cipher = DES.new(key_bytes, DES.MODE_ECB)

# 2. Mã hóa 2 bản rõ
c1 = cipher.encrypt(p1)
c2 = cipher.encrypt(p2)

# 3. Chuyển đổi bản mã sang chuỗi nhị phân (64 bit)
# Dùng format '064b' để đảm bảo luôn đủ 64 bit, không bị cắt mất số 0 ở đầu
c1_bin = format(int.from_bytes(c1, 'big'), '064b')
c2_bin = format(int.from_bytes(c2, 'big'), '064b')

# 4. Đếm số bit khác nhau (Tính khoảng cách Hamming)
diff_bits = 0
for b1, b2 in zip(c1_bin, c2_bin):
    if b1 != b2:
        diff_bits += 1

# 5. Tính tỷ lệ % bit bị thay đổi
percent_changed = (diff_bits / 64) * 100

print(f'--- Đang test với Key: {key_bytes} ---')
print(f'Bản mã 1 (p1): {c1_bin}')
print(f'Bản mã 2 (p2): {c2_bin}')
print(f'Số bit khác nhau: {diff_bits}/64')
print(f'Tỷ lệ thay đổi: {percent_changed:.2f}%\n")

# Chạy thử nghiệm 1: Key mặc định của đề bài
avalanche_test(b'87654321')

# Chạy thử nghiệm 2: Thay key bằng Mã số sinh viên (MSSV)
# LƯU Ý: Khóa DES BẮT BUỘC phải dài 8 bytes.
avalanche_test(b'24522019') # Hãy sửa lại thành MSSV của bạn và nhóm bạn!

```

Mục tiêu và Mô tả bài toán

- **Mục tiêu:** Kiểm chứng và đo lường "Hiệu ứng thác đổ" (Avalanche Effect) trên hệ mật mã khối DES. Hiệu ứng này đảm bảo rằng một thay đổi rất nhỏ ở đầu vào (bản rõ hoặc khóa) sẽ dẫn đến sự thay đổi lớn và không thể đoán trước ở đầu ra (bản mã).
- **Dữ liệu đầu vào:** * Bản rõ 1 (P_1): STAYHOME
 - Bản rõ 2 (P_2): STAYHOMA
- **Phân tích sự khác biệt:** Phân tích dưới dạng mã ASCII, ký tự cuối cùng 'E' (nhị phân: 01000101) và 'A' (nhị phân: 01000001) chỉ khác biệt nhau đúng **1 bit** (tại vị trí

bit thứ 3 từ phải sang). Tức là, trên tổng số 64 bit của khối dữ liệu, P_1 và P_2 giống hệt nhau 63 bit và chỉ sai lệch 1 bit.

Quá trình thực hiện

1. Khởi tạo thuật toán DES ở chế độ ECB (Electronic Code Book) bằng thư viện pycryptodome.
2. Tiến hành mã hóa P_1 và P_2 với cùng một khóa để thu được hai bản mã tương ứng (C và C').
3. Chuyển đổi C_1 và C_2 từ định dạng bytes sang chuỗi nhị phân 64-bit.
4. Tính toán khoảng cách Hamming (Hamming Distance) bằng cách duyệt và đếm số lượng bit khác biệt ở cùng vị trí giữa hai bản mã. Tính tỷ lệ phần trăm bit bị thay đổi.
5. Lặp lại thí nghiệm với hai trường hợp khóa: Khóa mặc định (87654321) và Khóa tự chọn (Mã số sinh viên).

Kết quả khi chạy:

```

...\\TH\\Lab02_ModernCryptography  P main ?  14:49
> python .\\task2_3.py
--- Đang test với Key: b'87654321' ---
Bản mã 1 (p1): 101000011100010011000001001100010101111100001100000010100110101
Bản mã 2 (p2): 1110111000010011111010110000011000110000000111111001011010111110
Số bit khác nhau: 37/64
Tỷ lệ thay đổi: 57.81%

--- Đang test với Key: b'24522019' ---
Bản mã 1 (p1): 0101000001111101110000010111110001111000100001011100101001001111
Bản mã 2 (p2): 1010001010110101100111001010100001000100000101111011100110010110
Số bit khác nhau: 34/64
Tỷ lệ thay đổi: 53.12%
```

Phân tích và Nhận xét:

Khả năng khuếch đại: Đầu vào chỉ có sự thay đổi cực kỳ nhỏ là 1 bit (chiếm tỷ lệ $\approx 1.56\%$ của khối 64-bit), nhưng kết quả đầu ra lại ghi nhận số lượng bit bị lật (flip) rất lớn, dao động xấp xỉ ở mức 50%. Sự khuếch đại này duy trì tính ổn định ngay cả khi thay đổi khóa mã hóa.

Nguyên lý hoạt động: Trong thuật toán DES, sự khuếch đại này không xảy ra ngay lập tức mà là hệ quả của cấu trúc Feistel trải qua 16 vòng lặp. Tại mỗi vòng, sự khác biệt 1 bit ban đầu bị xáo trộn và lan truyền rộng rãi sang các bit khác thông qua hàm F (đặc biệt là nhờ sự thay thế phi tuyến tính của các hộp S-box) và phép hoán vị.

Ý nghĩa mật mã học: Mức thay đổi $\approx 50\%$ là con số lý tưởng trong thiết kế mật mã. Nó chứng tỏ rằng mỗi bit của bản mã dường như được quyết định ngẫu nhiên (như việc tung đồng xu xác suất 50/50), không để lộ bất kỳ mối tương quan tuyến tính nào với bản rõ. Nhờ đạt được Hiệu ứng thác đổ (Avalanche Effect) mạnh mẽ, DES có khả năng chống lại các kỹ thuật thám mã vi phân (Differential Cryptanalysis) một cách hiệu quả.

Nhiệm vụ 2.4

SEED Lab - Secret-Key Encryption Lab (Task 5): Sinh viên thực hiện các bước sau bằng Python để hiểu rõ về đặc tính lan truyền lỗi (error propagation) của các chế độ mã hóa khác nhau:

1. Tạo dữ liệu 1000 byte.
2. Mã hóa dữ liệu bằng AES-128.
3. Làm hỏng bản mã bằng cách đảo 1 bit tại byte thứ 26.
4. Giải mã bản mã đã bị lỗi và *quan sát có bao nhiêu khối dữ liệu bản rõ bị hỏng với chế độ mã hóa* được sử dụng lần lượt là ECB, CBC, CFB, hoặc OFB

Mục tiêu thực nghiệm Nhiệm vụ này (dựa trên bài thực hành SEED Lab) nhằm mục đích kiểm tra và hiểu rõ về đặc tính lan truyền lỗi (error propagation) của các chế độ hoạt động (Mode of Operation) khác nhau trong mật mã khối. Cụ thể, bài kiểm tra đánh giá mức độ phá hủy đối với bản rõ khi bản mã bị nhiễu (hỏng) một lượng dữ liệu rất nhỏ trên đường truyền.

Thiết lập thông số thực nghiệm

- Thuật toán sử dụng: AES-128 (kích thước khối - block size là 16 bytes).
- Dữ liệu đầu vào: Tạo một tập dữ liệu bản rõ có kích thước 1000 byte. Sau khi thực hiện đệm (padding) chuẩn PKCS7 để chia hết cho 16, tổng kích thước dữ liệu là 1008 bytes, tương đương với 63 khối.
- Can thiệp lỗi (Injected Error): Sau khi mã hóa toàn bộ dữ liệu, tiến hành đảo (flip) đúng 1 bit tại byte thứ 26 của bản mã.
- Xác định vị trí lỗi: Vì mỗi khối gồm 16 bytes (Khối 1 từ byte 1-16, Khối 2 từ byte 17-32), nên byte thứ 26 nằm hoàn toàn bên trong Khối bản mã số 2.
- Kiểm tra: Tiến hành giải mã bản mã đã bị lỗi và đếm số lượng khối bản rõ bị hỏng trên 4 chế độ: ECB, CBC, CFB và OFB.

Source code:

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

# 1. Tạo dữ liệu 1000 byte (sử dụng ký tự 'A' để dễ quan sát)
plaintext = b'A' * 1000
# Đệm dữ liệu để chia hết cho 16 (AES block size). Kết quả sẽ là 1008 bytes (63 khối)
padded_plaintext = pad(plaintext, AES.block_size)

key = b'1234567890123456'
iv = b'1234567890123456'

# Danh sách các chế độ cần kiểm tra
modes = {
    'ECB': (AES.MODE_ECB, False),
    'CBC': (AES.MODE_CBC, True),
    'CFB': (AES.MODE_CFB, True),
    'OFB': (AES.MODE_OFB, True)
}

print("--- KIỂM TRA SỰ LAN TRUYỀN LỖI (ERROR PROPAGATION) ---")

for mode_name, (mode_val, use_iv) in modes.items():
    # 2. Mã hóa dữ liệu bằng AES-128
```

```

if use_iv:
    # Với CFB, set segment_size=128 để hoạt động theo chuẩn Block cipher 16-byte
    if mode_name == 'CFB':
        cipher = AES.new(key, mode_val, iv, segment_size=128)
    else:
        cipher = AES.new(key, mode_val, iv)
else:
    cipher = AES.new(key, mode_val)

ciphertext = bytearray(cipher.encrypt(padded_plaintext))

# 3. Làm hỏng bản mã: Đảo 1 bit tại byte thứ 26
# Vì byte đếm từ 1, byte thứ 26 nằm ở index 25.
# Byte 26 nằm gọn trong Khối số 2 (Khối 1: byte 1-16, Khối 2: byte 17-32)
ciphertext[25] ^= 0x01

# 4. Giải mã bản mã đã bị hỏng
if use_iv:
    if mode_name == 'CFB':
        dec_cipher = AES.new(key, mode_val, iv, segment_size=128)
    else:
        dec_cipher = AES.new(key, mode_val, iv)
else:
    dec_cipher = AES.new(key, mode_val)

decrypted = dec_cipher.decrypt(bytes(ciphertext))

# 5. Quan sát xem có bao nhiêu khối bị hỏng
corrupted_blocks = []
# Tổng số khối = 1008 / 16 = 63 khối
for i in range(len(padded_plaintext) // AES.block_size):
    start = i * AES.block_size
    end = start + AES.block_size

    # So sánh từng khối giải mã với khối gốc ban đầu
    if decrypted[start:end] != padded_plaintext[start:end]:
        corrupted_blocks.append(i + 1) # Cộng 1 để in ra Khối 1, Khối 2... dễ hiểu

print(f'Chế độ {mode_name:3}: Các khối bị hỏng là Khối {corrupted_blocks}')

```

Kết quả giải mã và Phân tích:

```

python .\task2_4.py
--- KIỂM TRA SỰ LAN TRUYỀN LỖI (ERROR PROPAGATION) ---
Chế độ ECB: Các khối bị hỏng là Khối [2]
Chế độ CBC: Các khối bị hỏng là Khối [2, 3]
Chế độ CFB: Các khối bị hỏng là Khối [2, 3]
Chế độ OFB: Các khối bị hỏng là Khối [2]

```

Dựa vào kết quả thu được từ chương trình, ta có phân tích cho từng chế độ như sau:

- **Chế độ ECB (Electronic Codebook):**
 - **Kết quả:** Khối bị hỏng là **Khối [2]**.
 - **Phân tích:** ECB mã hóa và giải mã từng khối một cách hoàn toàn độc lập. Việc Khối bản mã 2 bị sai 1 bit khi đi qua hàm giải mã AES sẽ khiến toàn bộ Khối bản rõ 2 bị biến thành dữ liệu rác. Do không có tính dây chuyền, lỗi chỉ dừng lại ở khối chứa nó và không lan truyền sang bất kỳ khối nào khác.
- **Chế độ CBC (Cipher Block Chaining):**
 - **Kết quả:** Các khối bị hỏng là **Khối [2, 3]**.
 - **Phân tích:** Trong chế độ giải mã CBC, Khối bản mã 2 được dùng ở hai nơi. Thứ nhất, nó đi qua hàm giải mã và làm hỏng hoàn toàn Khối bản rõ 2. Thứ hai, bản thân Khối bản mã 2 lại được dùng để thực hiện phép XOR với kết quả giải mã của Khối bản mã 3 để tạo ra Khối bản rõ 3. Vì Khối bản mã 2 bị sai 1 bit, nên Khối bản rõ 3 cũng sẽ bị sai đúng 1 bit ở vị trí tương ứng. Mức độ lan truyền lỗi ở CBC là hỏng khối hiện tại và 1 bit của khối tiếp theo.
- **Chế độ CFB (Cipher Feedback):**
 - **Kết quả:** Các khối bị hỏng là **Khối [2, 3]**.
 - **Phân tích:** Ở quá trình giải mã CFB, Khối bản mã 2 (lỗi 1 bit) được XOR với chuỗi khóa (keystream) sinh ra Khối bản rõ 2 bị sai đúng 1 bit. Tuy nhiên, Khối bản mã 2 lỗi này sau đó lại được đưa làm đầu vào cho thuật toán mã hóa để tạo ra chuỗi khóa giải mã cho Khối bản mã 3. Vì đầu vào bị sai, chuỗi khóa sinh ra bị sai hoàn toàn, dẫn đến việc Khối bản rõ 3 biến thành rác.
- **Chế độ OFB (Output Feedback):**
 - **Kết quả:** Khối bị hỏng là **Khối [2]**.
 - **Phân tích:** OFB hoạt động như một hệ mật mã luồng thực thụ. Chuỗi khóa (keystream) được tạo ra liên tục từ việc mã hóa Vector khởi tạo (IV) và hoàn toàn độc lập với bản rõ hay bản mã. Khối bản mã 2 bị lỗi 1 bit khi XOR với chuỗi khóa chuẩn xác sẽ chỉ làm sai đúng 1 bit tại Khối bản rõ 2. Do bản mã không tham gia vào việc tạo chuỗi khóa cho các khối sau, lỗi được cô lập hoàn toàn và không có tính lan truyền.

Nhiệm vụ 2.5

Tìm hiểu và so sánh độ mạnh và sự khác biệt của DES, Triple-DES (3DES), và AES. Tại sao chúng ta không nên sử dụng mã hóa double-DES (2DES)?

- **DES (Data Encryption Standard):**

Cơ chế: DES hoạt động dựa trên cấu trúc **Mạng Feistel**. Chia các khối dữ liệu thành 64 bit rồi chia 64 bit đó thành 32 bit trái và 32 bit phải. Ở mỗi vòng (round), DES sẽ giữ nguyên 32 bit bên Phải. 32 bit bên Phải này sẽ được đưa vào một "hàm biến đổi" (Function F) cùng

với **khóa con** của vòng đó. Kết quả sau khi biến đổi sẽ được đem đi cộng (XOR) với 32 bit nửa Trái rồi swap vị trí của 32 bit trái với 32 bit phải.

- **Triple-DES (3DES)**

Cơ chế: Thay vì mã hóa một lần, 3DES thực hiện quy trình: **Mã hóa -> Giải mã -> Mã hóa** bằng cách sử dụng 3 khóa DES khác nhau. Dữ liệu gốc được mã hóa bằng thuật toán DES với chìa khóa K1. Sau đó được đưa vào hàm *Giải mã* của DES nhưng lại dùng chìa khóa K2. Kết quả đó được mã hóa một lần nữa bằng DES với chìa khóa là K3.

- **AES (Advanced Encryption Standard)**

Cơ chế: AES không chia đôi dữ liệu như DES mà xử lý toàn bộ khối dữ liệu **128 bit** cùng một lúc. Nó sắp xếp 128 bit này vào một cái bảng (ma trận) **4x4**, trong đó mỗi ô là 1 byte (8 bit).

Tùy vào độ dài của khóa đã chọn thì AES sẽ thực hiện số vòng nhào trộn dữ liệu khác nhau:

- ❖ **AES-128:** 10 vòng.
- ❖ **AES-192:** 12 vòng.
- ❖ **AES-256:** 14 vòng.

Mỗi vòng gồm 4 bước:

Bước 1: SubBytes (Thay thế Byte)

Mỗi byte trong ma trận sẽ được thay thế bằng một byte khác dựa trên một bảng tra cứu cố định gọi là **S-Box**.

Bước 2: ShiftRows (Dịch chuyển hàng)

Các hàng trong ma trận 4x4 sẽ được đẩy đi ngang theo quy luật:

Hàng 1: Giữ nguyên.

Hàng 2: Đẩy sang trái 1 vị trí.

Hàng 3: Đẩy sang trái 2 vị trí.

Hàng 4: Đẩy sang trái 3 vị trí.

Bước 3: MixColumns (Trộn cột)

AES sử dụng phép nhân ma trận để trộn tất cả các byte trong cùng một cột lại với nhau.

Bước 4: AddRoundKey (Cộng khóa vòng)

Ở bước cuối mỗi vòng, một "khóa con" (được sinh ra từ chìa khóa chính) sẽ được đem đi XOR trực tiếp với ma trận dữ liệu hiện tại.

So sánh độ mạnh và sự khác biệt của DES, Triple-DES (3DES), và AES:

Đặc điểm	DES	3DES	AES
Độ dài khóa	56-bit	112 hoặc 168-bit	128, 192, 256-bit
Kích thước khối	64-bit	64-bit	128-bit
Số vòng lặp	16 vòng	48 vòng (16 x 3)	10, 12, hoặc 14 vòng
Tốc độ	Nhanh	Rất chậm	Rất nhanh
Độ an toàn	Đã bị phá giải	Đang bị khai tử	Tiêu chuẩn an toàn hiện nay

Tại sao chúng ta không nên sử dụng mã hóa double-DES (2DES)?

2DES không hề an toàn hơn DES vì một kiểu tấn công nổi tiếng gọi là Meet-in-the-middle attack làm cho độ khó phá mã chỉ có 2^{57} trường hợp.

Cơ chế tấn công: Giả sử mã hóa dữ liệu qua 2 bước

Dữ liệu gốc -> (Khóa 1) -> Kết quả trung gian -> (Khóa 2) -> Dữ liệu đã mã hóa.

Thay vì thử 2^{112} trường hợp thì kẻ tấn công sẽ thực hiện các cách sau:

Mã hóa xuôi: Thử tất cả các khả năng của Khóa 1 từ dữ liệu gốc và lưu toàn bộ "Kết quả trung gian" vào một bảng tra cứu lớn.

Giải mã ngược: Lấy dữ liệu đã mã hóa, thử giải mã bằng tất cả khả năng của Khóa 2.

So khớp: Nếu kết quả giải mã ngược trùng với bất kỳ kết quả nào trong bảng "Kết quả trung gian" ở bước 1, kẻ tấn công đã tìm ra cả Khóa 1 và Khóa 2.

Nhiệm vụ 2.6 (*)

Viết một chương trình (bằng ngôn ngữ lập trình của bạn) để đáp ứng các yêu cầu sau:

1. Số nguyên tố:

- Tạo ra một số nguyên tố ngẫu nhiên có 8 bits, 16 bits, 64 bits
- Xác định 10 số nguyên tố lớn nhất nhỏ hơn số nguyên tố Mersenne thứ 10.
- Kiểm tra xem một số nguyên tùy ý nhỏ hơn $2^{89} - 1$ có phải là số nguyên tố hay không?

2. Xác định ước số chung lớn nhất (GCD) của 2 số nguyên lớn tùy ý (lớn nhất có thể mà bạn có thể xử lý).

3. Tính toán lũy thừa theo module $ax \bmod p$. Chương trình của bạn phải có khả năng tính toán trong trường hợp số mũ lớn ($x > 40$), ví dụ, $7^{40} \bmod 19$

Source code:

```
import sympy
import random
import math

print("=== 1. KIỂM TRA VÀ TẠO SỐ NGUYÊN TỐ ===")

# 1.1 Tạo số nguyên tố 8, 16, 64 bits
def generate_prime(bits):
    # Dải giá trị cho số n-bit là từ  $2^{(n-1)}$  đến  $(2^n - 1)$ 
    lower = 2**(bits - 1)
    upper = 2**bits - 1
    return sympy.randprime(lower, upper)

print(f"Số nguyên tố ngẫu nhiên 8-bit : {generate_prime(8)}")
print(f"Số nguyên tố ngẫu nhiên 16-bit: {generate_prime(16)}")
print(f"Số nguyên tố ngẫu nhiên 64-bit: {generate_prime(64)}\n")

# 1.2 Xác định 10 số nguyên tố lớn nhất nhỏ hơn số nguyên tố Mersenne thứ 10
# Số nguyên tố Mersenne có dạng  $2^p - 1$ . Số Mersenne thứ 10 ứng với  $p = 89$ 
M10 = (2**89) - 1
print(f"Số nguyên tố Mersenne thứ 10 (M10) = {M10}")

primes_before_M10 = []
```

```

current = M10
for _ in range(10):
    current = sympy.prevprime(current) # Tìm số nguyên tố liền trước
    primes_before_M10.append(current)

print("10 số nguyên tố lớn nhất nhỏ hơn M10:")
for i, p in enumerate(primes_before_M10, 1):
    print(f" {i}. {p}")

# 1.3 Kiểm tra một số nguyên tùy ý nhỏ hơn  $2^{89} - 1$ 
test_num = random.randint(2, M10 - 1)
# Hàm sympy.isprime sử dụng thuật toán Miller-Rabin dưới nền
is_prime = sympy.isprime(test_num)
print(f"\nKiểm tra số ngẫu nhiên: {test_num}")
print(f"Kết quả (Miller-Rabin) : {'LÀ số nguyên tố' if is_prime else 'KHÔNG phải số nguyên tố'}")

print("\n=== 2. ƯỚC CHUNG LỚN NHẤT (GCD) ===")
# Sử dụng thuật toán Euclid thông qua hàm math.gcd
num1 = random.randint(10**50, 10**60) # Số cực lớn ~ 60 chữ số
num2 = random.randint(10**50, 10**60)
gcd_result = math.gcd(num1, num2)
print(f"Số A: {num1}")
print(f"Số B: {num2}")
print(f"Ước chung lớn nhất GCD(A, B): {gcd_result}")

print("\n=== 3. LŨY THỪA MODULO (MODULAR EXPONENTIATION) ===")
a, x, p = 7, 40, 19
result = pow(a, x, p)
print(f"Ví dụ đề bài: {a}^{x} mod {p} = {result}")

```

Mục tiêu thực nghiệm Nhiệm vụ này nhằm mục đích ôn tập và cài đặt các kiến thức nền tảng về lý thuyết số, phục vụ trực tiếp cho quá trình tạo khóa và mã hóa/giải mã của hệ mật mã bất đối xứng RSA. Các phép toán cốt lõi bao gồm: khởi tạo số nguyên tố lớn, thuật toán kiểm tra tính nguyên tố, tìm ước số chung lớn nhất (GCD) và tính lũy thừa modulo.

Kết quả chạy code:

```

> python .\task2_6.py
=== 1. KIỂM TRA VÀ TẠO SỐ NGUYÊN TỐ ===
Số nguyên tố ngẫu nhiên 8-bit : 251
Số nguyên tố ngẫu nhiên 16-bit: 59369
Số nguyên tố ngẫu nhiên 64-bit: 9888491802081282631

Số nguyên tố Mersenne thứ 10 (M10) = 618970019642690137449562111
10 số nguyên tố lớn nhất nhỏ hơn M10:
1. 618970019642690137449562091
2. 618970019642690137449562081
3. 618970019642690137449562063
4. 618970019642690137449562043
5. 618970019642690137449562013
6. 618970019642690137449562009
7. 618970019642690137449561847
8. 618970019642690137449561791
9. 618970019642690137449561671
10. 618970019642690137449561509

Kiểm tra số ngẫu nhiên: 386326398653444008780061324
Kết quả (Miller-Rabin) : KHÔNG phải số nguyên tố

=== 2. ƯỚC CHUNG LỚN NHẤT (GCD) ===
Số A: 638929121301608412235019713113176912843539296306596769250293
Số B: 293220161421731850859462634669476820834784982403869907500051
Ước chung lớn nhất GCD(A, B): 211

=== 3. LŨY THỪA MODULO (MODULAR EXPONENTIATION) ===
Ví dụ đề bài:  $7^{40} \bmod 19 = 7$ 

```

Sinh số nguyên tố n-bit: Sử dụng thư viện *sympy*, chương trình đã sinh ngẫu nhiên thành công các số nguyên tố với độ dài bit tăng dần:

- 8-bit: 149
- 16-bit: 47881
- 64-bit: 17669418456050459279

Khám phá số nguyên tố Mersenne: Số nguyên tố Mersenne thứ 10

Kiểm tra tính nguyên tố: Chương trình đã lấy ngẫu nhiên số 192130187163736984980477122 (nhỏ hơn M10) và áp dụng thuật toán kiểm tra xác suất Miller-Rabin. Kết quả trả về khẳng định đây **KHÔNG** phải là số nguyên tố. Thuật toán Miller-Rabin cho phép kiểm tra tính nguyên tố của các số siêu lớn với tốc độ chớp nhoáng mà không cần phải thực hiện phân tích ra thừa số nguyên tố.

Tìm Ước chung lớn nhất (GCD)

- **Dữ liệu đầu vào:** Hai số nguyên ngẫu nhiên cực lớn (lên tới 60 chữ số):
 - Số A: 401301149845742337859149909190832379404051062254645913688661
 - Số B: 178000595346498043955381124250131201968867151865528557196791
- **Kết quả:** $\text{GCD}(A, B) = 3$.

Việc tìm ước chung lớn nhất cho hai số khổng lồ này được thực hiện tức thì nhờ áp dụng thuật toán Euclid (được tích hợp qua hàm `math.gcd()`). Trong thực tế RSA phép tính này đóng vai trò sống còn trong việc chọn khóa công khai e sao cho

$$\text{GCD}(e, \phi(n)) = 1.$$

Lũy thừa Modulo (Modular Exponentiation)

Kết quả: Chương trình đã tính toán thành công $7^{40} \bmod 19 = 7$

HẾT